Trustworthy Hardware Timestamps in Zephyr: Event Association, Clock Continuity, and Service Availability

Jonathan Yantis *Independent researcher*

1 November 2026 Research draft for author review

Abstract

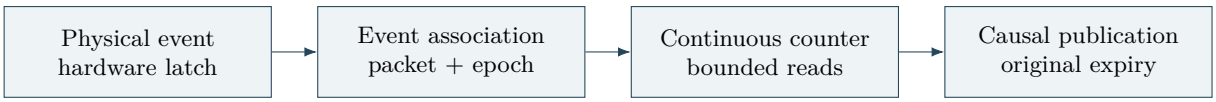
A hardware capture can be precise yet unusable because it belongs to another packet, a discontinued counter, or a clock model evaluated with stale processing time. This paper examines those integration problems in PToB's Zephyr/nRF52840 timing implementation. It describes controller-owned radio event association, one continuously extended capture domain, bounded snapshot readers, independent continuity checks, and coherent source evaluation. A concrete publication-order counterexample explains how newly published PPS or GNSS metadata can appear to be future input and interrupt qualification. These mechanisms build on established synchronization, timestamping and concurrency techniques; the contribution is an inspectable implementation account and a falsifiable evaluation method. Historical relative-PPS evidence is retained with its full population: 436 usable observations among 602 nominal opportunities, including invalid references, expired maps and missing witnesses. No new comparative hardware experiment is reported. The paper gives practical contracts and tests that can inform Zephyr's evolving precision-event infrastructure while keeping capture resolution, delivered agreement, service availability and calibrated accuracy distinct.

1 The engineering problem and the contribution

Zephyr applications can obtain nanosecond representations without having a nanosecond observation of the event they care about. A callback timestamps software execution; a radio capture timestamps a selected peripheral event. Neither value alone establishes packet identity, clock continuity, reference delay or source permission. Improving a servo cannot repair an observation taken in the wrong coordinate.

The case study is the T096 nRF52840 implementation at repository revision `a9ae0294cd8f`, using pinned Zephyr 4.1.0 dependencies [1]. Its relevant capture and publication sources are identified in the companion edition manifest. This source examination is separate from the older timing population discussed in Section 6. The implementation assumes one serialized counter-extension owner and a pinned single-core interrupt/resource configuration. It does not establish portability to arbitrary controllers, SMP, or zero-latency interrupt contexts.

The practical contribution has three parts: an event-to-publication contract, a source-ordering counterexample with existing production-body regressions, and a measurement plan that prevents lower error from concealing greater service loss. Comparative timing and execution-cost measurements remain proposed work. Earlier PToB papers already describe causal publication and relative timing [19, 20]; this account concentrates on the Zephyr integration boundaries and their failure modes.



Each boundary may refuse an observation. Retain the reason and the missing opportunity.

Copyright © 2026 Jonathan Yantis. Original paper, LaTeX and figure: CC BY 4.0. Illustrative pseudocode: MIT. Referenced code retains its own license. Author-review candidate; not submitted, peer reviewed or approved for library inclusion.

2 Prior art and the current Zephyr boundary

2.1 Established techniques, different endpoints

FTSP combined low-level timestamps, skew estimation and multihop distribution [7]. CheepSync brought low-level timestamps and compensation to BLE advertisers [8]. BlueSync studied beacon-based synchronization and efficient tick adjustment [9]. Nordic's 2016 implementation used timers and programmable peripheral interconnect (PPI) to synchronize proprietary radio beacons alongside BLE [10]. These works precede any claim based simply on a captured radio event, regression or a broadcast software clock.

Related systems also constrain the energy and operating-system claims. Virtual High-resolution Time combined fast active and slow sleeping clocks [13]. MicroSync combined hybrid timers and BLE communication scheduling; its high-accuracy configuration reports 272 ns mean absolute error and 319.2 ns standard deviation at 252.1 μW [11]. FLOPSYNC-3 introduced an RTOS virtual-clock interface and addressed the control consequences of applying correction at driver level [14]. These results have distinct references and statistics and are not ranked against PToB's PPS-map RMS.

Comparator	Established mechanism or endpoint	Remaining question for this study
Nordic timer demo [10]	PPI, hardware timers, proprietary beacons	Correct association inside the Zephyr BLE controller lifecycle
Maiwald et al. [18]	Periodic-advertising multicast; reported maximum reaction difference 11 μs	Capture-domain and publication behavior, rather than application reaction alone
Zhou et al. [12]	MPSL proprietary beacons; RSSI, traffic, cadence and connection-interval sweeps	Causal faults, false rejection and service coverage under matched workloads
Linux timestamping [16]	Delayed TX timestamp joined to original packet identity	Small, bounded implementation on a constrained RTOS
Linux sequence counters [15]	Consistent reads; explicit warning about preempted writers	Reader refusal cost and interrupt-context constraints

The 2026 Zhou preprint adapts Nordic's demo and measures physical GPIO differences under BLE traffic [12]. Its timing beacons use proprietary radio mode, whereas this case study observes supported BLE periodic PDUs. A new workload sweep or a smaller selected average would therefore be insufficient differentiation. The proposed evaluation isolates event association, lifecycle and publication ordering. The Maiwald comparison above uses the author institution's abstract; its full experimental method was not available in this review.

2.2 Build on the upstream mechanisms

Zephyr merged shared precision-time arithmetic, clock operations, a PI controller and a PTP adapter on 28 September 2026 [2, 4]. Its experimental subsystem leaves protocol policy and source handling to consumers. The broader architecture RFC already discusses counters, timestamp services and timescales [3]. The PTP/gPTP roadmap is developing these boundaries incrementally [5].

At this review date, draft PR 121471 provides scheduled output and PPS lifecycle work [6]. The roadmap's October input-capture discussion identifies IRQ ownership, bounded callbacks and low-rate polling as concrete requirements [5]. The documented periodic-advertising receive-info structure does not expose a physical hardware timestamp [17]; this observation concerns that interface, not every vendor controller API. Nordic's connected-time sample already exchanges anchor timestamps and event counters [21]. PToB's useful opportunity is to supply a supported BLE capture case, failure fixtures and measured costs to this existing work.

3 Associate the latch before interpreting the time

3.1 One counter, managed ownership

The examined adapter requests the high-frequency clock through Zephyr's clock manager and uses a 32-bit TIMER3 at 16 MHz [1]. The tick is 62.5 ns. PPS, radio ADDRESS and optional USB start-of-frame (SOF) capture into separate channels of that counter. A shared origin avoids a software bridge between those latches, but each peripheral still has its own endpoint delay and semantics.

Resource checks matter as much as the timer frequency. The adapter excludes the pinned controller's timer/PPI resources and leaves GPIOTE interrupt ownership with Zephyr. It does not commandeer the controller's RADIO or CLOCK tasks. A different SoC or controller configuration needs a new resource and interrupt audit; naming another spare timer is not sufficient.

3.2 An event record is an association, not just a scalar

The radio path retains the capture epoch, raw latch, controller owner, event counter, RF channel, direction and complete supported advertising bytes. Arm and completion hooks connect the latched event to the selected controller operation. Completion verifies the expected ADDRESS condition, completion status, receive CRC where applicable and accepted PDU shape before queue publication. The selected path is BLE 1M and rejects unsupported or ambiguous packet forms [1].

Listing 1: Illustrative contract; not a replacement driver.

```
arm(owner, pdu, event, channel, capture_epoch)
  require(supported_path && no_conflicting_capture)
  enable_owned_hardware_route()

complete(owner, done, crc)
  require(owner_matches_current_armed_operation)
  raw = read_latched_ADDRESS()
  disable_owned_route()
  require(done && ADDRESS_seen && receive_crc_ok_if_needed)
  require(supported_complete_PDU && queue_has_space)
  publish_copy(raw, epoch, owner, event, channel, direction, PDU)
```

An owner pointer is not a durable generation identifier: an allocator can reuse its address. Event counters wrap, and a controller can restart a train. Retain the counter epoch and the higher-level source/boot/path context; serialize closure and flush so old queued records cannot be admitted to a new lifecycle. If a future generic API uses an explicit generation token, document exactly which transition changes it.

3.3 Close the operation before reusing its resources

The capture queue has 16 entries. Queue overflow loses an observation; it does not authorize overwriting a timestamp and pretending the next packet supplied it. Disable refuses to report a clean close while a selected event remains armed. Stop/restart must resolve actual ownership and queued history before a new owner is admitted.

Raw ADDRESS is the present endpoint convention. TX and RX detector equivalence remains uncalibrated. Subtracting a PHY prefix can produce a register START coordinate, but does not measure an electrical launch or remove receiver delay. Keep raw capture, any normalization and calibration profile separate. A CRC establishes decoded packet integrity; it does not establish timestamp accuracy.

For delayed correction schemes, carry the original event identity through to the correction. Processing a later correction can update a map, but cannot refresh the original source permission simply by arriving later. A model's fit-reference instant, availability/activation instant and the physical event being evaluated are distinct. Past-event scoring must use a model available at that event, or explicitly report that causal reconstruction is unavailable [19, 20].

4 Extend the fine counter without inventing continuity

4.1 Separate counter extension from event reconstruction

Let r_i be a current 32-bit latch and C_i its extended value. With validated continuity,

$$d_i = (r_i - r_{i-1}) \bmod 2^{32}, \qquad C_i = C_{i-1} + d_i.$$

At 16 MHz the full wrap period is 268.435456 s. Unsigned subtraction alone cannot distinguish a short interval from that interval plus whole wraps. The serialized owner therefore requires timely extension, checks coarse monotonic progress and refuses excessive gaps, ambiguous deltas or integer overflow [1]. Its one-second continuity policy is much shorter than the hardware wrap.

For an older captured event r_e, reconstruct against a validated *current* latch r_n:

$$a = (r_n - r_e) \bmod 2^{32}, \qquad C_e = C_n - a.$$

The examined GM reader requires a matching epoch, a current base no more than 16,000,000 ticks behind the latch, and event age no greater than 4,000,000 ticks (250 ms). Check subtraction and extended range before conversion. Event age must be measured against current time, independently of the age of the last extension publication. An old base can otherwise make an already old queued event appear young.

4.2 Bound the reader; keep one extension owner

On a 32-bit MCU a 64-bit counter publication can tear. The GM reader obtains a sequence-validated group of 32-bit atomic fields: extended low/high words, raw base, epoch and continuity guard. It makes at most two attempts, then returns unavailable. It never becomes another wrap-extension owner.

Linux documents the familiar failure of an unbounded sequence-counter reader that interrupts its writer and spins waiting for that writer [15]. Bounded attempts adapt that established consistency pattern to a deadline-sensitive reader; they are not a new synchronization primitive. They bound this retry loop, not the whole call graph or memory-access latency. Writer serialization, memory ordering, compiler output and interrupt context still require platform-specific review.

The independent RTC1 observation is a negative continuity lease. It can reject an HF base whose extender has stalled; it does not supply fractional phase or replace a missing fine event. Its modular check relies on the accompanying extension/watchdog assumptions. A stopped or invalidated epoch remains closed even if an old publication arrives late. A new epoch requires an explicit lifecycle start.

4.3 Bracket cross-domain reads and state their aperture

Where kernel monotonic time and the HF domain must be related, the implementation brackets the HF read with kernel reads under a bounded IRQ critical section. Kernel reads are quantized: if q_K bounds their downward quantization, the HF sample's kernel coordinate is enclosed by $[K_L, K_H + q_K)$ under the stated ordering assumptions. The bridge rejects a reported width above its 100 μs policy ceiling and restores the prior IRQ key on every path.

That ceiling is a refusal rule, not a measured normal duration or an RF error bound. The permission converter additionally shortens remaining lifetimes using a 200 ppm rate assumption and a 32 μs quantization margin; its header identifies the rate assumption as unverified. This bridge is not a calibrated phase map. Measure achieved aperture and interrupt blocking under workload. Its serialization assumptions exclude competing zero-latency access to the latch.

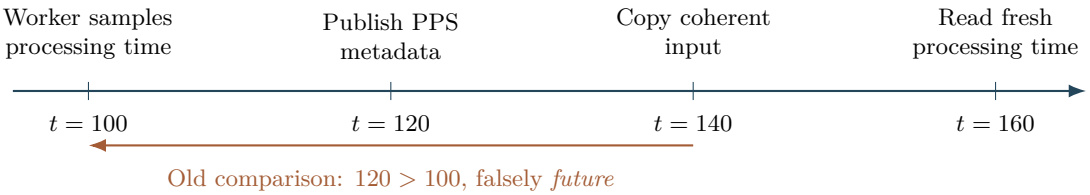
A disciplined map may be written $T = aC + b$, with an identified source and counter epoch. Nanosecond arithmetic improves its representation; it does not reduce the 62.5 ns capture grid or prove a physical uncertainty bound. Unknown reference, path or detector terms must remain unknown even when regression residuals are small.

5 A freshness race caused by publication ordering

5.1 New input can look like future input

Freshness checks normally reject a sample timestamp t_s when the processing time $t_q < t_s$. That is correct if both belong to one domain and t_q was obtained after observing the sample. A stale loop timestamp violates that ordering assumption.

Consider the following synthetic schedule, in microseconds:



The metadata is coherent and the PPS is current, yet evaluation against 100 rejects it. Repeating that pattern can reset source qualification or withdraw service. Reducing timer quantization or loosening a PLL gain cannot repair the ordering error. Increasing the permitted sample lifetime would hide it while weakening genuine expiry checks.

5.2 Copy input, refresh the evaluation clock, preserve expiry

The examined service copies source metadata and its physical map coherently. While holding that source lock, it obtains a fresh validated bridge and returns that same bridge to subsequent decisions [1]. This orders the copied sample before the evaluation time while preventing concurrent publication from changing the copied state.

Listing 2: Fresh evaluation time does not grant a fresh source lifetime.

```
lock(source_owner)
  snapshot = copy_coherent_metadata_and_source_map()
  now = read_validated_kernel_HF_bridge()
unlock(source_owner)
require(now.valid && snapshot.counter_epoch == now.epoch)

require(snapshot.sample_time <= now.in_sample_domain)
require(now.in_sample_domain < snapshot.original_expiry)
return evaluate(snapshot, now)   // retain source and capture generations
```

The real production routine also evaluates the physical source at a later same-epoch capture and retains the distinction between its sample instant and the earlier bridge. The pseudocode states the ordering invariant, not exact executable instructions. Authentication, stop/fatal state and source generation remain separate gates. The original PPS/GNSS timestamps and their expiry horizons are not renewed by copying them.

5.3 A regression with a narrow, useful claim

The existing `gm7_source_order_test.c` constructs PPS and RMC publication between an earlier bridge and the coherent copy. Its 11 current cases include genuine future PPS, exact freshness expiry, invalid capture domain, epoch replacement, stopped/fatal source and unavailable UTC association. The runner can also accept an explicitly selected historical source fixture and demonstrate the old ordering failure [1].

This is a controlled production-body counterexample with fake kernel/capture boundaries. It identifies a possible software failure chain; it does not establish the physical cause of a historical field withdrawal. This draft does not report a fresh run of that native regression. The archive replay in Section 6 is a different check.

The broader rule is to acquire coherent state before the time used to judge it, then revalidate the applicable lifecycle and expiry. Apply it to watchdog snapshots, USB source maps and status publications as well as GNSS. Keep system scheduling on its monotonic clock; a synchronized or civil-time adjustment should not silently change timeout semantics.

6 Evidence, measurement and an adoptable artifact

6.1 Historical precision with its denominator intact

The selected 12 October 2026 archive evaluates a causal software map at local physical PPS after two BLE links. Its 436 reference-valid usable observations have signed mean −18.417431 ns, bias-inclusive RMS 62.503871 ns, absolute P99 160.25 ns and maximum 191 ns [19]. The observation span is 601.5761865 s. The conservative nominal count is:

$$602 = \underbrace{436}_{\text{usable, reference valid}} + \underbrace{65}_{\text{invalid reference}} + \underbrace{3}_{\text{expired map, reference valid}} + \underbrace{98}_{\text{unobserved}}.$$

All 501 available-map causal observations, including invalid references, have RMS 3.929464 ms. That separate population preserves excursions; it is not a valid-reference accuracy estimate. Usable observed coverage is 72.425% of nominal opportunities. Missing witnesses are unknown service intervals, not proof of an outage.

The historical calibration was a fixed empirical +10,543 ns per received link. Its training inputs are absent from the public archive, so that calibration cannot be independently regenerated there. The experiment neither measures an output pin nor establishes calibrated UTC accuracy, and it cannot qualify the current GM7 source. Preparation reran `python3tools/verify_v14.py`: the public numerical populations and archive checks passed. No new RF acquisition or physical timing comparison accompanied this draft.

6.2 A controlled comparison that can establish benefit

Use matched devices, cadence and predeclared observation windows. Compare callback timestamps, ISR timer reads, hardware capture, and the complete guarded path. Keep deliberately broken lifecycle/order variants in controlled fixtures. Test physical configurations only with their separately established operating scope.

Question	Controlled intervention	Required observation
Capture benefit	Callback versus ISR versus hardware latch	Signed bias, RMS, P99/maximum and scoring coverage
Association integrity	Delay, cancellation, owner reuse, overflow	Wrong acceptance, correct refusal and retained gaps
Continuity	Missed extension, wrap, stop/restart, epoch change	Alias prevention, recovery and reader attempts
Freshness ordering	Publish between old *now* and input copy	False future rejection and qualification resets
Operating cost	USB, display, logging and BLE control load	Aperture, IRQ blocking, queue occupancy and availability

For every scheduled query, record the service verdict and witness eligibility separately. Report returned-valid availability, independently scoreable coverage and conditional error statistics. A conservative observed within-budget fraction is

$$\frac{\#\{\text{valid output, eligible witness, } |e| \le \epsilon\}}{\#\{\text{all scheduled opportunities}\}}.$$

Unknown opportunities remain identified alongside that fraction; they are not proven failures or successes. Withholding every difficult sample cannot count as improved timing service. Measure energy only with an actual power instrument, and freeze calibration before held-out runs or devices.

6.3 Practical conclusion and remaining work

Package a small capture example, explicit clock/event/epoch contracts, controlled failure fixtures, source pins and complete denominators. Retain the pinned 4.1 baseline and evaluate a separately pinned current Zephyr revision before claiming an upstream deficiency. Attribute the existing Zephyr/ZMK backports; successful compilation or simulator checks add no physical qualification.

The present result is an inspectable engineering account, not a new synchronization primitive. Its useful next contribution is to demonstrate how much correct event association, continuity checks and publication ordering improve the accuracy–availability–cost tradeoff. That evidence could inform Zephyr's ongoing event-capture work and give application engineers a reproducible route from a hardware latch to usable time.

References

[1] J. Yantis, *PToB T096 source and dependency lock*, repository revision `a9ae0294cd8f`, 1 Nov. 2026. Repository. Companion `edition.json` identifies the examined files; code licenses are preserved. This review does not assert that the local revision is a published release.

[2] Zephyr Project, *Precision Timing*, experimental subsystem documentation. Documentation.

[3] Zephyr Project, *RFC: Proposal for a Precision Clock Subsystem Architecture*, issue 76335, opened 2024. Architecture RFC.

[4] Zephyr Project, *Reusable precision timing infrastructure for PTP and gPTP*, PR 115363, merged 28 Sept. 2026. Merged implementation.

[5] Zephyr Project, *RFC: Roadmap for PTP/gPTP*, issue 107837, 2026; including the 23 Oct. input-capture use case. Roadmap and discussion.

[6] Zephyr Project, *Scheduled output and PPS service*, draft PR 121471, 2026. Proposed implementation.

[7] M. Maroti, B. Kusy, G. Simon and A. Ledeczi, *The Flooding Time Synchronization Protocol*, SenSys, pp. 39–49, 2004. doi:10.1145/1031495.1031501.

[8] S. Sridhar, P. Misra, G. S. Gill and J. Warrior, *CheepSync: A Time Synchronization Service for Resource Constrained Bluetooth Low Energy Advertisers*, 2015; revised 2016. arXiv:1501.06479.

[9] F. Asgarian and K. Najafi, *BlueSync: Time Synchronization in Bluetooth Low Energy With Energy-Efficient Calculations*, IEEE Internet of Things Journal, 9(11), pp. 8633–8645, 2022; online 2021. doi:10.1109/JIOT.2021.3116921. Publisher abstract reviewed.

[10] Nordic Semiconductor (Audun), *Wireless timer synchronization among nRF5 devices*, 12 July 2016. Vendor article; implementation and NCS sample.

[11] R. Ohara et al., *MicroSync: Sub-Micro Second Accuracy Wireless Time Synchronization Service*, IEEE Access, 12, pp. 124478–124494, 2024. doi:10.1109/ACCESS.2024.3446668; version of record.

[12] Z. Zhou et al., *Multiprotocol Wireless Timer Synchronization for IoT Systems*, author preprint, 8 Apr. 2026. arXiv:2604.07199.

[13] T. Schmid, P. Dutta and M. B. Srivastava, *High-Resolution, Low-Power Time Synchronization: an Oxymoron No More*, IPSN, 2010. Author-hosted paper.

[14] A. Sorrentino, F. Terraneo and A. Leva, *Efficient Abstraction of Clock Synchronization at the Operating System Level*, NG-RES, pp. 4:1–4:11, 2023. doi:10.4230/OASIcs.NG-RES.2023.4.

[15] Linux Kernel, *Sequence counters and sequential locks*. Kernel documentation.

[16] Linux Kernel, *Timestamping*, including delayed TX timestamp/packet association. Kernel documentation.

[17] Zephyr Project, *bt_le_per_adv_sync_recv_info*, API documentation identified as 4.5.0-rc3. Receive-report fields.

[18] T. Maiwald et al., *Timing Deviations in Wireless Networks using Bluetooth Periodic Advertising*, I2MTC, pp. 1–6, 2022. doi:10.1109/I2MTC48687.2022.9806632; author institution abstract reviewed.

[19] J. Yantis, *V14 successor research: 12 October 2026*, selected evidence, summary and verifier. Evidence collection; `tools/verify_v14.py`. Historical application 0.14.43; no current-firmware acceptance implied.

[20] J. Yantis, *PToB Viability: Rooted BLE Timing Meshes for Heterogeneous Edge Networks*, research draft, 30 Oct. 2026. Manuscript; see also the separate 12 Oct. boundary-clock study and 24 Oct. ESP32-S3 implementation guide in the repository paper catalogue.

[21] Nordic Semiconductor, *Bluetooth: Connection time synchronization*, nRF Connect SDK documentation. Connection-anchor sample.

Review boundary. Online sources were reviewed on 1 November 2026. Development documentation, issue bodies and PR status may change. This is a targeted primary-source review, not a proof of priority. The companion source ledger records access limitations. Codex assisted with research, drafting and artifact checks; Jonathan Yantis is the author responsible for review and any later publication.